IMPLEMENTATION BRIEF · JUL 06 — JUL 11

Token-Efficient Agent.

A hybrid cost-optimizing inference router — **local-first, Fireworks-fallback.**
Answers 19 tasks. Clears the 80% gate. Spends near-zero tokens.

CONSTRAINTS

4 GB
RAM

2 vCPU
COMPUTE

10 min
WALL-CLOCK

0 GPU
ACCELERATOR

The rules of the game.

A general-purpose agent must answer **19 fixed tasks across 8 capability categories** — under a two-stage gate and a strict token-cost leaderboard.

STAGE 01

Accuracy Gate

Must clear **80% (16 / 19)** — or be excluded from the leaderboard entirely.

ERROR BUDGET → 3 wrong answers

STAGE 02

Leaderboard Rank

Ranked by **Fireworks tokens spent** — lower is better.

TARGET → as close to zero as possible

HARD CONSTRAINT

10-minute execution window · 4 GB RAM · 2 vCPU · no GPU · no pre-installed runtime

WINNING PLAY

Clear the gate with margin, then spend as few tokens as possible.

MAJOR CLARIFICATION · JUL 09

Local-first. Fireworks-fallback.

Local-model answers **count toward the accuracy gate** — but cost zero Fireworks tokens. That single rule inverts the entire competition.

OLD DOCTRINE

Every answer from Fireworks. Minimize prompts per call.

```
tokens_spent = Σ  f(task)  ·  for all 19 tasks
floor = 19  ·  min_call_cost
```

▲ WHAT MOST TEAMS ARE STILL BUILDING

NEW DOCTRINE

Local is free. Fireworks is fallback.

```
tokens_spent = Σ  f(task)  ·  only if local fails
floor → 0  ·  theoretical optimum
```

▲ OUR LANE

THE WHOLE STRATEGY, IN ONE LINE

Clear 80% gate with margin → win by answering max tasks locally at zero tokens → reserve Fireworks for tasks a small model can't clear.

Eight categories.
Nineteen tasks.

01 / FACTUAL

Knowledge

Concise explanations & definitions

→ ~2-3 TASKS

02 / MATH

Reasoning

Multi-step calculations; correct final value

→ ~2-3 TASKS

03 / SENTIMENT

Classification

Label + short justification

→ ~2-3 TASKS

04 / SUMMARY

Summarisation

Condense to stated format / length

→ ~2-3 TASKS

05 / NER

Named Entities

Person · Org · Location · Date

→ ~2-3 TASKS

06 / DEBUG

Code Debugging

Bug identified + corrected code

→ ~2-3 TASKS

07 / LOGIC

Deductive

Answer satisfying all constraints

→ ~2-3 TASKS

08 / CODE GEN

Generation

Correct function from spec

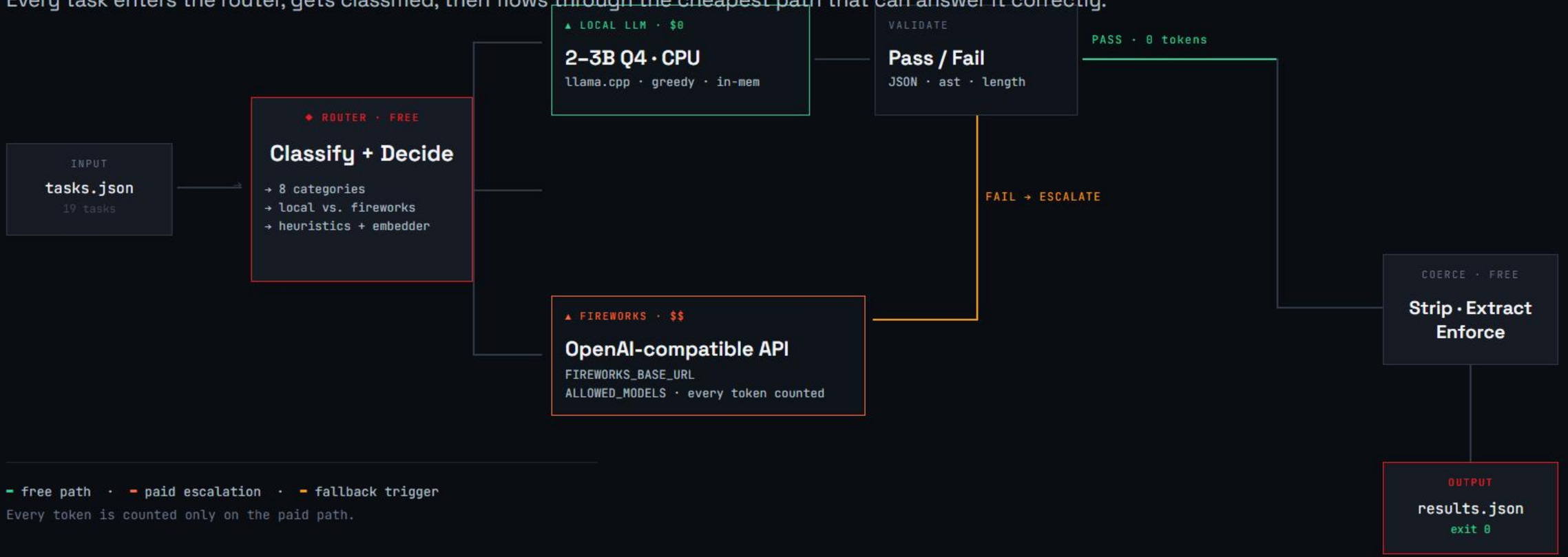
→ ~2-3 TASKS

ERROR BUDGET

Each category appears ~2–3 times · 3 wrong answers = we're out.

One pipeline. Two paths.

Every task enters the router, gets classified, then flows through the cheapest path that can answer it correctly.



free path · paid escalation · fallback trigger

Every token is counted only on the paid path.

The free tier.

Two components do the heavy lifting at zero token cost — router + bundled local LLM, both CPU-only.

A · ROUTER

$0 · CPU

Classifier & gatekeeper.

Classifies each task into one of 8 categories and decides **local vs. Fireworks** based on measured per-category policy.

TIER A	Pattern matching — code fences, "summarize", "sentiment", etc.
TIER B	Optional tiny ONNX embedder for ambiguous cases

B · LOCAL LLM

$0 · CPU

Gemma-3-4b-it Q4_K_M.

Bundled 2–3B 4-bit GGUF model. Runs on CPU via `llama-cpp-python`. Loaded once at startup, kept in memory.

1.5–2 GB	< 30 s	T = 0	$0 / call
RAM FOOTPRINT	PER-CALL CAP	GREEDY / DETERMINISTIC	TOKEN COST

▲ HARD-CAPPED MAX_TOKENS · PER-CALL WALL-CLOCK GUARD

Strategies, Fireworks, coercion.

The paid escalation path, the shape of every prompt, and the free post-processing that keeps us from re-asking.

C

Per-category strategies.

Templates defining prompt, model choice, **max_tokens**, stop sequences — for **both** paths.

```
local → simple · fast · bundled
fires → terse · zero-shot · minimal
```

E.G. SENTIMENT = LOCAL · MATH = FIRES-FIRST

D

Fireworks client.

OpenAI-compatible wrapper. Retries with exponential backoff. Own token telemetry for tuning.

ALLOWED_MODELS

- minimax-m3
- kimi-k2p7-code
- gemma-4-31b-it
- gemma-4-26b-a4b-it
- gemma-4-31b-it-nvfp4

E

Validate + coerce.

Free, local. The reason we don't re-ask — we fix output ourselves.

CHECKS

- JSON parses?
- Code ast.parses?
- Within length limit?

✕ fail → escalate to Fireworks

Who answers what.

Not guessed — measured by our calibration harness. Per category × path × output policy.

CATEGORY	DEFAULT ANSWERER	IF ESCALATED	OUTPUT POLICY
Factual	Local · escalate on low conf.	`gemma-4-26b`	1–3 sentences
Math	Fireworks-first	`gemma-4-31b`	Final answer only
Sentiment	Local	`gemma-4-26b`	Label + reason
Summarisation	Local (short) · Fires (long)	`gemma-4-31b-nvfp4`	Obey length exactly
NER	Local	`gemma-4-26b`	Structured list
Code Debugging	Fireworks-first	`gemma-4-31b → kimi`	Code + 1-line cause
Logic	Fireworks-first	`gemma-4-31b → minimax-m3`	Minimal reasoning
Code Generation	Local (simple) · Fires (complex)	`gemma-4-31b → kimi`	Function only

ESCALATION LADDER

`route → local answer → validate → (fail) → Fireworks: cheapest model → validate`

Eight ways to bleed less.

Only escalated tasks cost tokens. For those, we compress every axis simultaneously.

01 **Answer locally whenever the gate allows.**
The single biggest token win. Everything else is second-order.

02 **Gemma-first on escalation.**
Non-reasoning · terse · 262k vocab = fewest tokens per answer.

03 **Terse or absent system prompt.**
Fold instructions into a compact per-category template.

04 **Zero-shot by default.**
Add examples only where the eval harness proves they help.

05 **Stop sequences + tight max_tokens.**
`\n\n · code-fence · first period.`

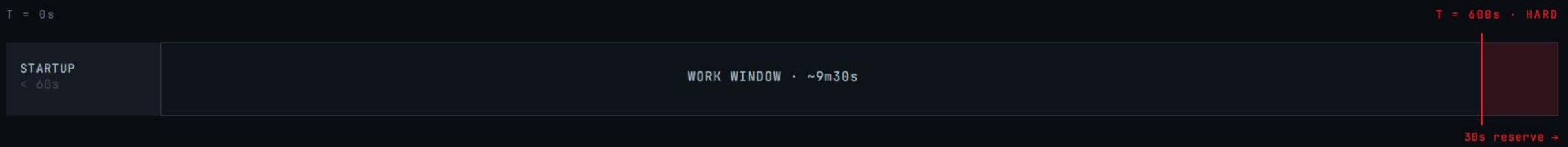
06 **Local output coercion, not re-asking.**
Strip preambles · extract answer · enforce length — all free.

07 **Suppress reasoning / CoT traces.**
Where accuracy allows — CoT is a 3–5× token tax.

08 **Tokenizer arbitrage.**
All Gemmas share one tokenizer — identical token count across sizes.

The clock is the boss.

Timeout = no output = worse than any token bill. We spend tokens to buy time.



LOCAL · SERIAL

2 vCPU saturated by one llama.cpp call.

Effectively one inference at a time. Bounded per-task cap: < 30s.

FIRES · PARALLEL

Network-bound async pool, bounded.

Escalations run concurrently — parallelism amortizes latency spikes.

TIME-VS-TOKENS POLICY

If falling behind: stop starting local inferences → push remaining tasks to fast Fireworks calls. Always write valid JSON + exit 0 by the hard deadline.

Never crash. Never timeout.

Every known failure mode has an owner and a mitigation baked into the pipeline.

OPERATING ENVELOPE

Startup	< 60 s
Per-task latency	< 30 s
Model size cap	≤ 3B · 4-bit
Image size	< 10 GB
Runtime host	4 GB · 2 vCPU · no GPU
Exit code discipline	exit 0 = success

FAILURE MODE · MITIGATION

TIMEOUT
Deadline guard + time-vs-tokens policy

INVALID_RESULTS_SCHEMA
Validate output schema before write

MODEL_VIOLATION
ALLOWED_MODELS only, via FIREWORKS_BASE_URL

IMAGE_TOO_LARGE
Slim base + 4-bit weights · target < 10 GB

TASK CRASH
Per-task try / except — one failure never crashes the run

COMPETITIVE EDGE

Most teams guess.
We measure.

01
Synthetic dev set
~30–50 tasks/category + gold answers. Shaped like the real 19-task mix; includes adversarial variants.

02
Proxy LLM-judge
Strong model scoring intent-match — proxy for the hidden accuracy gate.

03 NEW
Local calibration sweep
Answer locally only if judge pass-rate ≥ ~90%. Produces the policy table **empirically**, not by guess.

04
Strategy sweep · escalated
For each escalated category × Fireworks model × policy × max_tokens: pick **token-minimal** config that clears gate with margin.

05
Wall-clock model
Measure real tokens/sec on 2 vCPU; project if N local inferences fit in 10 min; tune local↔Fireworks tradeoff.

06
Two operating modes
`safe` → more Fireworks, guaranteed
`lean` → max local, tokens → 0

▲ EACH MECHANISM CONVERTS A GUESS INTO A NUMBER

Ten submissions per hour.
Use them.

The leaderboard is our second eval loop — always retreat to a pinned safe baseline if accuracy drops below 80%.

PHASE 01 — DAY 1 · SUB #1

Pure-Fireworks safe baseline.

- Confirm I/O contract, Docker, env
- Validate 80% gate clears
- Pin this as fallback config

OUTCOME → SAFETY NET EXISTS

PHASE 02 — DAY 2-3 · SUB #2-4

Layer in local answering.

- Enable local for proven categories
  sentiment → NER → easy factual
- Verify accuracy holds
- Re-enable only when calibration says so

OUTCOME → TOKENS DROP, ACCURACY HOLDS

PHASE 03 — DAY 4-5 · SUB #5+

Push toward zero tokens.

- Move more categories local
- Retreat to safe if accuracy dips
- Final lean push before freeze

OUTCOME → TOKENS → 0

One image. Self-contained.

Dockerfile — linux/amd64

```
FROM    python:3.12-slim
ARG     --platform=linux/amd64

# bundled model weights (~1.5-2 GB)
COPY    models/gemma-3-4b-it-Q4_K_M.gguf  /app/models/

# CPU-only wheel — no CUDA
RUN     pip install --no-cache-dir llama-cpp-python
RUN     apt-get clean && rm -rf /var/lib/apt/lists/*

# env-only — no .env, no hardcoded keys
ENV     FIREWORKS_API_KEY     = <from platform>
ENV     FIREWORKS_BASE_URL    = <from platform>
ENV     ALLOWED_MODELS        = <from platform>

# I/O contract
CMD     ["python", "-m", "agent",
         "--in", "/input/tasks.json",
         "--out", "/output/results.json"]
```

SHIPPING TARGETS

Image size (compressed)	< 10 GB
Runtime RAM	≤ 4 GB
Compute	2 vCPU · no GPU
Model ceiling	≤ 3B · Q4
Registry	GHCR · public
License	MIT · OSS

I/O CONTRACT

```
read  /input/tasks.json
write /output/results.json
```

Six days.

Jul 6 → Jul 11 · shipping something submittable by end of Day 1, then compounding.

Kickoff

- Confirm unknowns
- Deploy Gemma on Fires
- Download local model
- Dev key ready

Safe baseline

- Pure Fireworks end-to-end
- Docker build + push
- **First leaderboard submit**

Bundle local

- Wire local tier
- Router + validation
- Eval harness + judge

Calibrate

- Local calibration sweep
- **Lock policy table**
- Enable safe categories

Push local

- More categories local
- Strategy sweeps
- Harden deadline guard
- Robustness

Final lean push

- Freeze & regression test
- Presentation + video
- SUBMIT < 21:30 IST

▲ EACH DAY ENDS WITH A SUBMITTED IMAGE ON THE LEADERBOARD

Why this wins.

01

Clearer thinking than the field.

The Jul 9 clarification inverted conventional wisdom: **local answers count.** Most teams are still building "every call → Fireworks."

→ COMPETITIVE ADVANTAGE FROM READING RULES CAREFULLY

02

Measured, not guessed.

Eval harness + proxy judge → empirical policy table. Adaptable: if Gemma deployment slows, swap the model, not the doctrine.

→ DATA WHERE OTHER TEAMS HAVE INTUITION

03

Reliability by design.

Deadline guard = no timeouts. Per-task try/except = no crashes. Local + Fires balance keeps accuracy above the gate.

→ VALID OUTPUT + EXIT 0, EVEN IN WORST CASE

04

A story judges can pitch.

"Hybrid cost-optimizing inference router." Runs on a 2-vCPU/4GB box, stays accurate, spends near-zero on most tasks. LLM cost at the edge.

→ APP OF TECH + ORIGINALITY + BUSINESS VALUE

Definition of done.

Every number is monitored throughout the loop — miss any and we ship the pinned safe config instead.

METRIC	TARGET	WHY IT MATTERS
Accuracy	≥ 90% · 17–18 / 19	Clear 80% gate with margin; judge is nondeterministic
Fireworks tokens	→ 0	Leaderboard ranking is entirely determined by token spend
Wall-clock	< 10 min total	Hard deadline — 60s startup + ~9m30s work
Image size	< 10 GB compressed	Avoid IMAGE_TOO_LARGE failure
Startup time	< 60 s	Includes model load — leaves time for actual work
Per-task latency	< 30 s	Per-request rule; prevents timeout on any single task
Code quality	exit 0 + valid JSON	Always write output. Never crash. No exceptions.

In one paragraph.

Build a containerized general-purpose agent that routes every task locally (free), answers as many as it reliably can with a bundled 2–3B 4-bit local model at zero Fireworks-token cost, and escalates to a single, tightly-constrained Gemma-first Fireworks call only for tasks a small model can't clear — validating and coercing every output locally.

A time-aware deadline guard guarantees valid output within 10 minutes and **spends tokens rather than timing out.** An eval harness with a proxy judge calibrates, per category, what we can answer locally while staying comfortably above the 80% gate — iterating on the real leaderboard from a pinned safe baseline toward an as-local-as-possible, near-zero-token lean optimum.

▲ LOCAL-FIRST. FIREWORKS-FALLBACK. TOKENS → 0.